

Assignment 1

Overview

The assignments in this course are designed as a single, evolving *project* that you will build throughout the semester. Instead of completing unrelated tasks each time, you will gradually extend and improve the same application as new concepts are introduced.

This mirrors how real software systems are developed: features are added over time, designs are refined, and earlier decisions matter later.

Each assignment focuses on a specific set of skills while intentionally limiting others. Technologies and features that are not allowed in early assignments will be introduced later in the course when the required background has been covered.

Data Storage

The data in assignment 1 will be stored in JSON files. Sample data files have been provided. You must make sure that you are not adding new fields or corrupting exiting ones. For example, a 'filename' (if required) property should never be renamed to 'file_name' and of course you should not erase any of the fields.

Your program must use the *fs/promises* functions `readFile` and `writeFile` (the use of `readFileSync` or `writeFileSync` or MongoDB is not permitted and will result in a decreased grade).

The files are as follows:

- `employees.json`: data about the employees:
 - `employeeId`: string that starts with 'E' has 3 digits (zero padded on the left)
 - `name`: string representing the full name
 - `phone`: string representing the phone number
- `shifts.json`: data about shift timings:
 - `shiftId`: string starting with 'S' and 3 digits.
 - `date`: A string representing the date stored in YYYY-MM-DD format.
 - `startTime`: A string representing the starting time of the shift in HH:MM format
 - `endTime`: A string representing the ending time of the shift in HH:MM format
- `assignments.json`: data about which employee has been assigned to which shifts
 - `employeeId`: a string representing the employee ID
 - `shiftId`: a string representing the shiftID

User Interface

For this assignment, you will be creating a menu-driven console application. The application will show the menu and allow the user to select an operation by typing a number. You will need to use `console.log()` to display information and the `prompt-sync` package for reading information from the user.

After each operation, you **MUST** show the menu again and ask the user for their next operation. You are not permitted to just run a single operation and stop. Selecting ‘Exit’ will terminate the program.

```
1. Show all employees
2. Add new employee
3. Assign employee to shift
4. View employee schedule
5. Exit
What is your choice>
```

Only the “happy path” use cases are described but you **MUST** handle errors appropriately by displaying messages; the program must not crash under normal operations.

List Employees

In this use case, you will be displaying a list of all employees currently loaded into the system from the JSON file. Note that you must load the list of employees each time that you print the list, do not load them at the start of the application and keep them in memory.

The output formatting is important, please note how the phone numbers are aligned correctly regardless of how long the name is. Please make sure that the formatting of your output is good. You may assume that names do not exceed 20 characters, but a better option is to look at all the names first and then adjust the width of the middle column to be just wide enough.

```
What is your choice> 1
Employee ID  Name                Phone
-----
E001        Alex Morgan         5555-0101
E002        Samir Khan          5555-0102
E003        Lina Perez          5555-0103
E004        Daniel Wu           5555-0104
E005        Fatima Noor         5555-0105
```

Add Employee

In this use case, you will need to:

1. Prompt for the name and phone number of the new employee.
2. Determine the “next” ID number like an auto-increment field in a relational database such as MySQL.
3. Write the new information to the employees.json file.

```
What is your choice> 2
Enter employee name: Mohammed
Enter phone number: 1234-5678
Employee added...
```

1. Show all employees
2. Add new employee
3. Assign employee to shift
4. View employee schedule
5. Exit

```
What is your choice> 1
```

Employee ID	Name	Phone
E001	Alex Morgan	5555-0101
E002	Samir Khan	5555-0102
E003	Lina Perez	5555-0103
E004	Daniel Wu	5555-0104
E005	Fatima Noor	5555-0105
E006	Mohammed	1234-5678

Assign Shift

In this use case, you will need to:

1. Ask for the employee id
2. Ask for the shift id.
3. Add the employee to the shift or provide an error message indicating that the assignment was not possible and why.

The constraints for this feature:

- You must ensure referential integrity on the employee id and the shift id. Think about what this means. See the sample run for a hint.
- The assignment “bridge table” needs to enforce a composite key. Think about what this means. See the sample run for a hint.

```
What is your choice> 3
Enter employee ID: E012
Enter shift ID: S001
Employee does not exist
```

```
What is your choice> 3
Enter employee ID: E002
Enter shift ID: S123
Shift does not exist
```

```
What is your choice> 3
Enter employee ID: E003
Enter shift ID: S007
Employee already assigned to shift
```

```
What is your choice> 3
Enter employee ID: E003
Enter shift ID: S008
Shift Recorded
```

View Employee Schedule

In this feature you will just ask for the employee ID and will generate an output that could be copy/pasted into a CSV file (do not actually generate the CSV file).

If there is no employee with the given ID or the employee exists without any shifts, then only the header will be displayed.

```
What is your choice> 4
Enter employee ID: E003
```

```
date,startTime,endTime
2026-01-06,15:00,21:00
2026-01-08,09:00,15:00
2026-01-08,15:00,21:00
```

Programming Style and Documentation

Please read this section carefully as it can negatively impact your grade.

A program in industry that works but is sloppy (bad indentation, inappropriate variable and function names, etc) causes overhead on the company because it takes additional time for other developers to find problems and to add new features to existing code. You are required to keep your code clean in order to receive the best possible grade for the assignment.

Code Guidelines (All Assignments)

- Indentation must be 4 spaces.
- Semi-colons are not required at the end of statements.
- Variables must be camel case.
- You must use function declarations: `function doSomething()`.
- You must NOT use arrow function declarations: `const doSomething = () => {}`
- Anonymous functions are not permitted for this assignment.
- Built-in array iteration methods such as `.forEach()`, `.map()`, `.filter()` and similar methods are not permitted. Use `for` and `while` loops to process arrays.
- Use JSDoc style comments for every function used. Comments in other part of the code are permitted but there is no need to document the obvious.

JSDoc Comments

For all assignments in this course, you are required to write comment blocks using syntax that is compatible with JSDoc (<https://jsdoc.app/>).

Here is an example of a function used to validate a string to ensure that it contains a valid date:

```
/**
 * Check a string to see if it is a valid date in the YYYY-MM-DD.
 *
 * @param {String} dateString The string to check.
 * @returns {Boolean} True/False depending if the input was considered to be valid.
 */
function isValidDate(dateString) {
```

If you run jsdoc (you are not required to but you are encourage), you will be able to generate some very nice HTML files such as:

`isValidDate(dateString) → {Boolean}`

Check a string to see if it is a valid date in the YYYY-MM-DD.

Parameters:

Name	Type	Description
<code>dateString</code>	String	The string to check.

Source: [assign1.js, line 95](#)

Returns:

True/False depending if the input was considered to be valid.

Type

Boolean

GitHub

We require the use of GitHub for assignments in this course. It is not mandatory for assignment 1 but it will be required for all future assignments. It is highly suggested that you start using GitHub for assignment 1 to detect permission errors and flow problems so that you do not get lower grades in future assignments.

- Please create an account on github.com if you do not have one.
- Create a private repository called `infs3201_winter2026`. All assignments will use the same repository. Only the assignments must be placed into the repository. If you want to use GitHub for labs, put them in a different project.
- Do not create any folder such as “assignment 1” in the repository. Assignments will be releases, not a folder.
- Invite your instructor to be a collaborator on the project. Check with your instructor about which email to use, but likely it will be their UDST email.

We do not use GitHub features such as pull requests, branching, and merging in this course. We use a simple clone, pull, commit, and push style of code management. We will use tags and releases. A short demonstration video is available on D2L (Student Resources) explaining how to use these features.

The submission of the assignment must be done by uploading a zip file to D2L for your submission. In addition, in the submission details box, include a link to your repository. If the link is not available to your instructor, points will be deducted (except for the first assignment where you will receive just a warning).

Development in real life is not a “single block of time where everything is completed”; development is iterative where parts of the software is developed and then later fixed and updated. Multiple commits during each assignment are expected. In future assignments,

specific commits will be explicitly indicated. In the absence of specific instructions, the following items are good:

- When you have some small feature working.
- You are about to take a break and want a backup.
- You fix a bug that you discovered.

Submission

Create a zip with everything including the JSON files but be warned that the instructor will use their own files to check your program. Include a link to the GitHub repository if you are using one.

Late Submissions

Late submissions will be accepted until January 31 at 11:59pm with a penalty of 50%, after which no submissions will be accepted.

Because multiple weeks have been allocated for the completion of the assignment, sick notes will not be accepted to either skip the assignment or extend the deadline.

Course Learning Outcome (CLO) Alignment

This section describes the alignment to the CLOs in the course. Students may ignore this section; its primary purpose is for accreditation review.

This assignment contributes to the following Course Learning Outcomes (CLOs):

CLO	Description	Alignment
1	Explore layered software architectures for web application implementation	Partial
2	Experiment with advanced server-side technologies	Limited
3	Develop server-side web applications	Strong
4	Apply appropriate design patterns and software architectures	Strong

Note: Assignment 1 focuses on foundational server-side logic, data modeling, and architectural concepts. Full web-based delivery, database systems, and advanced server-side technologies are introduced and assessed in later assignments.

Grading

Criteria	Exemplary	Satisfactory	Developing	Unsatisfactory
Program Features and Operation	<i>9 points</i> The program implements all features which work as expected. The user was not able to crash the program with inputs.	<i>7 points</i> The program is missing minor features or can cause crashes by providing specific inputs.	<i>5 points</i> The program is missing major functional features.	<i>0 points</i> The program cannot be run due to a large number of errors. The user is able to cause the program to fail with normal inputs.
Code Design	<i>4 points</i> Appropriate functions used with good parameters and return types.	<i>3 points</i> Some functions have issues in terms of parameters and return types.	<i>1 point</i> Functions were used but design was not appropriate.	<i>0 points</i> Functions were not used or parameters and return values were done using global variables.
Formatting and Documentation	<i>2 points</i> All functions have comments that allow JSDoc to be run indicating parameter and return types. Formatting of the code is excellent.	<i>1 point</i> Some functions have appropriate comments. JSDocs is not able to generate comment blocks. Formatting is mostly good but some areas could use improvement.	<i>0 points</i> Only basic comments provided. Many parts of the code require reformatting.	<i>0 points</i> Code has limited or no comments. Significant formatting issues.